

Manual de Selenium WebDriver 4

Table of Contents

1. ¿Qué es Selenium?
 - 1.1. Dependencia Maven
2. El WebDriver
 - 2.1. La interfaz `WebDriver`
 - 2.2. Cómo se crea una instancia
 - 2.3. Configurar el navegador al crearlo
 - 2.4. Cerrar el navegador
3. Localizar elementos
 - 3.1. Por atributos del elemento
 - 3.2. Búsquedas encadenadas
 - 3.3. XPath
 - 3.4. Selectores CSS
 - 3.5. Leer el resultado
4. Interactuar con elementos
 - 4.1. Listas desplegables (`<select>`)
 - 4.2. Checkboxes y radio buttons
5. Acciones avanzadas
 - 5.1. La clase `Actions`
 - 5.2. Ejecutar JavaScript
 - 5.3. Diálogos nativos (`Alert`)
 - 5.4. Cookies
6. Sincronización (esperas)
 - 6.1. Espera implícita (`implicitlyWait`)
 - 6.2. Espera explícita (`WebDriverWait` + `ExpectedConditions`)
 - 6.3. Espera flexible (`FluentWait`)
7. Page Object Model (POM)
8. Capturas de pantalla y eventos
 - 8.1. `TakesScreenshot`
 - 8.2. `WebDriverListener` : reaccionar a eventos del driver
9. Selenium Grid y ejecución remota
 - 9.1. Combinándolo con tests en paralelo
10. Buenas prácticas (y errores comunes)
11. Referencia rápida

Manual de referencia rápida sobre Selenium WebDriver 4. No documenta un ejercicio concreto: recoge, organizados por concepto, los mecanismos de Selenium más usados en el día a día, con ejemplos de código propios.

1. ¿Qué es Selenium?

Selenium automatiza un navegador real: abre páginas, busca elementos del DOM, hace clic, escribe texto, lee valores... exactamente las mismas acciones que haría una persona, pero desde código. Se usa sobre todo para tests **funcionales de extremo a extremo (E2E)**: comprobar que una aplicación web funciona de verdad, tal y como la usaría un usuario, en lugar de probar una clase aislada como en un test unitario.

1.1. Dependencia Maven

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.27.0</version>
</dependency>
```

XML

Un único artefacto trae la API de `WebDriver`, los bindings de cada navegador y las utilidades (`Actions`, `Select`, `ExpectedConditions` ...) que se ven en este manual.

2. El WebDriver

2.1. La interfaz `WebDriver`

`WebDriver` es la pieza central de todo el API: representa al navegador y es la **misma interfaz** independientemente de qué navegador haya detrás. Un test escrito contra `WebDriver` (localizar elementos, hacer clic, leer texto...) funciona igual con Firefox, Chrome o Edge — lo único que cambia es la clase concreta que se instancia para crearlo.

2.2. Cómo se crea una instancia

Crear un `WebDriver` es, en esencia, un proceso de dos pasos: decirle a Selenium **dónde está el ejecutable del driver** del navegador, y luego **instanciar la clase** de ese navegador.

```
System.setProperty("webdriver.gecko.driver",
    "src/test/resources/drivers/geckodriver/geckodriver.exe");
WebDriver driver = new FirefoxDriver();

driver.get("http://www.google.com");
...
driver.quit();
```

JAVA

Cada navegador necesita su propio **driver** (un ejecutable que traduce las llamadas de Selenium a comandos que el navegador entiende): `geckodriver` para Firefox, `chromedriver` para Chrome, `msedgedriver` para Edge...

`System.setProperty("webdriver.<navegador>.driver", ruta)` es la forma explícita de indicar dónde está ese ejecutable, y hay que llamarlo **antes** de instanciar el driver.



Desde Selenium 4.6, si no se fija esa propiedad, **Selenium Manager** intenta localizar (o descargar) automáticamente el driver adecuado para la versión de navegador instalada. Fijar la ruta a mano sigue siendo útil para tener una versión de driver reproducible en el equipo del curso, sin depender de una descarga en el momento de ejecutar el test.

Cada implementación de `WebDriver` corresponde a un navegador:

Clase	Navegador
FirefoxDriver	Firefox (usa geckodriver)
ChromeDriver	Chrome/Chromium (usa chromedriver)
EdgeDriver	Microsoft Edge (usa msedgedriver)
RemoteWebDriver	Cualquier navegador servido por un Selenium Grid remoto (ver Selenium Grid y ejecución remota)

Las cuatro implementan la misma interfaz `WebDriver`, así que a partir de aquí el resto del test (localizadores, esperas, aserciones) se escribe exactamente igual sin importar cuál se haya instanciado.

2.3. Configurar el navegador al crearlo

Cada navegador tiene además una clase `*Options` (`FirefoxOptions`, `ChromeOptions`, `EdgeOptions` ...) para configurar cómo debe arrancar, que se pasa al constructor del driver en vez de (o además de) crearlo sin argumentos:

```
FirefoxOptions options = new FirefoxOptions();
options.setCapability(CapabilityType.PLATFORM_NAME, Platform.WINDOWS);

WebDriver driver = new FirefoxDriver(options);
```

JAVA

Es el mismo mecanismo que se usa para pedir un navegador concreto a un Selenium Grid (ver Selenium Grid y ejecución remota), y también el sitio donde se configuran otras opciones habituales aunque no aparezcan en este ejercicio, como arrancar en modo **headless** (sin interfaz gráfica, típico en CI) o fijar el tamaño de la ventana.

2.4. Cerrar el navegador



`driver.quit()` cierra el navegador y libera todos los recursos de la sesión. Sin él, el proceso del navegador (y del driver) puede quedar abierto en segundo plano aunque el test haya terminado. Es habitual comentarlo temporalmente durante el desarrollo de un test para poder inspeccionar el navegador tras la ejecución, pero debe estar activo (normalmente en un `@AfterAll`) en cualquier test que se vaya a dejar corriendo de forma desatendida (CI, por ejemplo).

3. Localizar elementos

`driver.findElement(By...)` devuelve el **primer** elemento que coincide con el criterio de búsqueda; `driver.findElements(By...)` (plural) devuelve **todos** los que coinciden, en una `List<WebElement>` (vacía si no hay ninguno). Si `findElement` no encuentra nada, lanza `NoSuchElementException` — no hace falta comprobar `assertNotNull` después de un `findElement`: si ha devuelto algo, ya existe.

3.1. Por atributos del elemento

Localizador	Ejemplo
<code>By.id("...")</code>	<code>driver.findElement(By.id("user"))</code>

Localizador	Ejemplo
<code>By.name("...")</code>	<code>driver.findElement(By.name("username"))</code>
<code>By.className("...")</code>	<code>driver.findElement(By.className("in-user"))</code> — cuidado: si varios elementos comparten la clase, <code>findElement</code> solo devuelve el primero; para todos hay que usar <code>findElements</code> .
<code>By.tagName("...")</code>	<code>driver.findElements(By.tagName("label"))</code> — casi siempre en plural, para recorrer todos los elementos de una etiqueta.
<code>By.linkText("...")</code> / <code>By.partialLinkText("...")</code>	<code>driver.findElement(By.linkText("Página"))</code> busca un <code><a></code> por su texto exacto; <code>partialLinkText</code> por una parte de ese texto.

3.2. Búsquedas encadenadas

Un `WebElement` implementa la misma interfaz de búsqueda que `WebDriver`, así que se puede acotar una búsqueda a los descendientes de un elemento ya encontrado:

```
WebElement formulario = driver.findElement(By.id("loginForm"));
WebElement username = formulario.findElement(By.className("in-user"));
```

JAVA

Es más explícito que una única consulta XPath/CSS equivalente, aunque para casos sencillos suele ser más cómodo expresar directamente la jerarquía en el propio selector (ver más abajo).

3.3. XPath

XPath permite expresar rutas y condiciones sobre cualquier atributo, incluida la posición en el árbol:

Ejemplo	Significa
<code>By.xpath("/html/body/form/input")</code>	Ruta absoluta: primer <code><input></code> hijo directo del primer <code><form></code> , dentro del primer <code><body></code> , dentro de <code><html></code> .
<code>By.xpath("//input[@type='submit']")</code>	<code>//</code> busca en cualquier parte del documento: cualquier <code><input></code> cuyo atributo <code>type</code> sea <code>submit</code> .
<code>By.xpath("//input[@type='submit' and @value='Login']")</code>	Combinación de condiciones con <code>and</code> (también existe <code>or</code>).
<code>By.xpath("//img[not(@alt)]")</code>	Todas las <code></code> que no tengan atributo <code>alt</code> .
<code>By.xpath("//input[@*='password']")</code>	Cualquier <code><input></code> que tenga algún atributo (el que sea) con ese valor.

3.4. Selectores CSS

Casi siempre más cortos y legibles que el XPath equivalente, y habitualmente más rápidos de evaluar por el navegador:

Ejemplo	Significa
<code>By.cssSelector(html > body > form > div > label)</code>	<code>></code> es "hijo directo" (equivalente sin <code>></code> : "descendiente en cualquier nivel").
<code>By.cssSelector("input.in-user")</code>	Un <code><input></code> con esa clase.
<code>By.cssSelector("input#pass")</code> / <code>By.cssSelector("#pass")</code>	Por id (el segundo, sin restringir la etiqueta, es el habitual).
<code>By.cssSelector("input[name='password']")</code>	Por el valor de un atributo cualquiera.
<code>By.cssSelector("li:first-child")</code> / <code>li:nth-child(2)</code> / <code>li:last-child</code>	Selección por posición entre los hermanos.
<code>By.cssSelector("li + li")</code>	El <code></code> que va inmediatamente después de otro <code></code> (el segundo elemento de una lista).
<code>By.cssSelector("input:focus")</code> / <code>input:disabled</code>	Por estado del elemento (con foco, deshabilitado...).



Las herramientas de desarrollador del navegador (`F12`) son la forma más rápida de inspeccionar el HTML real y comprobar un selector antes de escribirlo en el test.

3.5. Leer el resultado

Método	Para qué
<code>element.getText()</code>	El texto visible del elemento (y sus descendientes).
<code>element.getDomAttribute("...")</code>	El valor de un atributo del elemento tal y como está en el HTML fuente (por ejemplo, <code>href</code> , <code>value</code> , <code>placeholder</code>).
<code>element.getCssValue("...")</code>	El valor calculado de una propiedad CSS (por ejemplo, <code>width</code> , <code>background-color</code>).
<code>element.isSelected()</code>	Si un checkbox/radio/opción está marcado.
<code>element.isDisplayed()</code>	Si el elemento es visible (existir en el DOM no implica ser visible).



Selenium 4 distingue `getDomAttribute` (el atributo tal cual aparece en el HTML) de `getDomProperty` (el valor actual de la propiedad en el DOM en vivo, que puede haber cambiado por JavaScript) — el antiguo `getAttribute` mezclaba ambos conceptos y hoy se considera impreciso; en código nuevo es mejor usar explícitamente el que corresponda.

4. Interactuar con elementos

Método	Para qué
<code>element.click()</code>	Simula un clic.
<code>element.sendKeys("...")</code>	Escribe texto (no borra lo que hubiera antes: hay que llamar antes a <code>clear()</code> si se quiere sustituir el contenido).
<code>element.clear()</code>	Vacía un campo de texto.
<code>element.submit()</code>	Envía el formulario al que pertenece el elemento (equivalente a pulsar Intro en un campo de texto dentro de un <code><form></code>).

4.1. Listas desplegables (`<select>`)

La clase `Select` envuelve un `WebElement` que sea un `<select>` y añade operaciones específicas:

```
Select coches = new Select(driver.findElement(By.id("carOptions")));

coches.isMultiple();           // ¿admite selección múltiple?
coches.getOptions().size();    // número de <option>

coches.selectByVisibleText("Tesla"); // por el texto que ve el usuario
coches.selectByValue("ford");       // por el atributo value
coches.selectByIndex(0);           // por posición

coches.getFirstSelectedOption(); // la opción seleccionada actualmente
```

JAVA

Si el `<select>` admite selección múltiple, además:

```
colores.selectByValue("red");           // añade una selección más
colores.getAllSelectedOptions();        // todas las opciones marcadas
colores.deselectByValue("yellow");
colores.deselectByIndex(0);
colores.deselectByVisibleText("Black");
colores.deselectAll();
```

JAVA

4.2. Checkboxes y radio buttons

No tienen una clase auxiliar propia: son `WebElement` normales, y se comprueban/cambian con `isSelected()` + `click()`:

```
WebElement checkMusica = driver.findElement(By.xpath("//input[@id='musica']"));
if (!checkMusica.isSelected()) {
    checkMusica.click();
}
assertTrue(checkMusica.isSelected());
```

JAVA



`click()` **alterna** el estado — no tiene un "marcar" o "desmarcar" explícito. Por eso el patrón siempre es comprobar `isSelected()` antes de decidir si hace falta el `click()`: si el test hiciera `click()` sin comprobar primero, ejecutarlo dos veces (por ejemplo, al repetir el test tras un fallo previo) dejaría el checkbox en el estado contrario al esperado.

5. Acciones avanzadas

5.1. La clase `Actions`

Para interacciones que no son un simple `click()` / `sendKeys()` — doble clic, arrastrar y soltar, mantener pulsada una tecla, mover el ratón... — Selenium ofrece un API de "acciones compuestas":

```
Actions actions = new Actions(driver);

actions.doubleClick(mensaje).perform();

actions.dragAndDrop(source, target).perform();
```

JAVA

Las acciones se **encadenan** sobre el mismo objeto `Actions` y no ocurren hasta que se llama a `.perform()` — hasta entonces solo se están describiendo los pasos a ejecutar.

5.2. Ejecutar JavaScript

`JavascriptExecutor` permite ejecutar cualquier script en el contexto de la página cargada — útil para invocar funciones JS propias de la aplicación, o como último recurso cuando algo no es accesible con el resto del API de Selenium:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("saludar()");
```

JAVA

5.3. Diálogos nativos (`Alert`)

Los `alert()` / `confirm()` / `prompt()` de JavaScript no forman parte del DOM de la página: hay que cambiar el foco de Selenium a ellos explícitamente con `driver.switchTo()`:

```
Alert alert = driver.switchTo().alert();
alert.accept();    // equivalente a pulsar "Aceptar"
// alert.dismiss(); // equivalente a "Cancelar"
// alert.getText();  // el texto mostrado
```

JAVA

5.4. Cookies

```
driver.manage().addCookie(new Cookie("cookie_selenium", "valor"));
Cookie c = driver.manage().getCookieNamed("cookie_selenium");
driver.manage().deleteCookieNamed("cookie_selenium");
```

JAVA

Útil, por ejemplo, para comprobar que un login ha creado la cookie de sesión esperada (`JSESSIONID` en una aplicación Java tradicional), sin depender de inspeccionar la interfaz.

6. Sincronización (esperas)

Una página web moderna no termina de "estar lista" en el instante en que `driver.get()` vuelve: puede haber peticiones AJAX en marcha, animaciones, elementos que aparecen tras un evento...Selenium ofrece tres mecanismos para esperar a que un elemento esté disponible, de peor a mejor:



La peor opción, y la más tentadora, es una espera activa con `Thread.sleep(...)`. Funciona "hasta que deja de funcionar": si la página tarda más de lo previsto (una vez, en una máquina más lenta, bajo carga...) el test falla igualmente, y si tarda menos, el test pierde tiempo esperando sin motivo. Los mecanismos de espera de Selenium existen precisamente para no depender de un número de milisegundos fijado a ojo.

6.1. Espera implícita (`implicitlyWait`)

Se configura una vez sobre el `driver` y afecta a **todas** las búsquedas posteriores: si un `findElement` no encuentra el elemento a la primera, reintenta hasta que aparece o hasta que se cumple el plazo.

```
driver.manage().timeouts().implicitlyWait(Duration.ofMillis(8000));

WebElement button = driver.findElement(By.cssSelector("#btn-1")); // espera hasta 8s si hace falta
```

JAVA

Es la forma más sencilla de activar, pero es "todo o nada": se aplica igual a cada búsqueda de toda la sesión, sin poder ajustar la condición de espera caso por caso.

6.2. Espera explícita (`WebDriverWait` + `ExpectedConditions`)

Detiene el test hasta que se cumple una condición concreta, con un límite máximo de tiempo:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(60));
wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("#btn-1")));

WebElement button = driver.findElement(By.cssSelector("#btn-1"));
button.click();
```

JAVA

`ExpectedConditions` trae condiciones ya hechas para los casos más comunes: presencia en el DOM, visibilidad, que sea clicable, que un texto aparezca, que un elemento desaparezca...

6.3. Espera flexible (`FluentWait`)

La variante más configurable: además del tiempo máximo, se controla cada cuánto se reintenta (`pollingEvery`) y qué excepciones se ignoran mientras se espera (por ejemplo, `NoSuchElementException`, que se produce constantemente mientras el elemento aún no existe):

```
FluentWait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofMillis(60000))
    .pollingEvery(Duration.ofMillis(250))
    .ignoring(Exception.class);

WebElement button = wait.until(d -> d.findElement(By.id("btn-1")));
```

JAVA

`wait.until(...)` recibe una función `WebDriver → T`: la lambda se reevalúa cada `pollingEvery` hasta que devuelve un valor que no es `null / false`, o hasta que se agota `withTimeout`.

Mecanismo	Se configura	Alcance
Implícita	Una vez sobre el driver	Todas las búsquedas de la sesión
<code>WebDriverWait</code>	Por cada espera	Una condición predefinida (<code>ExpectedConditions</code>)
<code>FluentWait</code>	Por cada espera	Una condición cualquiera (lambda), con <code>polling</code> e ignorado de excepciones configurables



No mezclar espera implícita con `WebDriverWait / FluentWait` sobre el mismo `driver`: el tiempo total de espera puede acabar sumando ambas de forma poco predecible. Lo habitual es elegir una única estrategia por proyecto (normalmente esperas explícitas) y no activar la implícita.

7. Page Object Model (POM)

Repetir los mismos `By.id(...)` / `By.cssSelector(...)` en cada test que toca un formulario es frágil: si cambia un `id` en el HTML, hay que corregirlo en todos los tests que lo usaban. El patrón **Page Object Model** propone encapsular, en una única clase, todos los elementos de una pantalla concreta:

```
public class FormularioPeliculaPage {  
  
    @FindBy(id = "btnInsertar")    private WebElement btnInsertar;  
    @FindBy(name = "titulo")       private WebElement tfTitulo;  
    @FindBy(name = "director")     private WebElement tfDirector;  
  
    public FormularioPeliculaPage(WebDriver driver) {  
        PageFactory.initElements(driver, this);  
    }  
  
    public WebElement getBtnInsertar() { return btnInsertar; }  
    public WebElement getTfTitulo()    { return tfTitulo; }  
    public WebElement getTfDirector()  { return tfDirector; }  
}
```

- `@FindBy` declara **cómo** localizar cada campo (por `id`, `name`, `className` ... los mismos criterios de Localizar elementos), pero no lo busca todavía — el campo se queda a `null`.
- `PageFactory.initElements(driver, this)`, en el constructor, es quien realmente rellena todos los campos anotados, de una vez.

El test queda así centrado en el **flujo**, no en el detalle de cómo se localiza cada campo:

```

@Test
void insertarPelicula() {
    driver.get("../formulario.html");
    FormularioPeliculaPage page = new FormularioPeliculaPage(driver);

    page.getTfTitulo().sendKeys("El último gran héroe");
    page.getTfDirector().sendKeys("John McTiernan");
    page.getBtnInsertar().click();

    // asertos sobre el resultado...
}

```

Si mañana cambia el `id` de `btnInsertar` en el HTML, solo hay que tocar una línea en la clase de página — ningún test cambia.

8. Capturas de pantalla y eventos

8.1. TakesScreenshot

```

TakesScreenshot ts = (TakesScreenshot) driver;
File srcFile = ts.getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(srcFile, new File("src/test/resources/screenshots/estado.png"));

```

`getScreenshotAs` devuelve la captura en memoria en el formato pedido (`OutputType.FILE` la deja como fichero temporal, `OutputType.BASE64 / BYTES` para incrustarla en un informe); hay que copiarla a una ruta propia si se quiere conservar.

8.2. WebDriverListener : reaccionar a eventos del driver

Para no tener que llamar a `getScreenshotAs` a mano en cada punto donde algo podría fallar, Selenium permite **decorar** el `WebDriver` con un listener que recibe eventos de cualquier operación (antes/después de un `findElement`, un `click`, un `get`, o cuando cualquier llamada lanza una excepción):

```

public class MiListener implements WebDriverListener {

    private final WebDriver driver;
    public MiListener(WebDriver driver) { this.driver = driver; }

    @Override
    public void beforeClick(WebElement element) {
        System.out.println("Antes de pulsar: " + element);
    }

    @Override
    public void onError(Object target, Method method, Object[] args,
        InvocationTargetException e) {
        File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
        FileUtils.copyFile(src, new File("src/test/resources/screenshots/onError.png"));
    }
}

```

Y se activa envolviendo el `driver` original:

JAVA

```
WebDriverListener listener = new MiListener(driver);
WebDriver decorated = new EventFiringDecorator<>(listener).decorate(driver);

decorated.get("https://es.wikipedia.org");           // dispara beforeGet/afterGet
decorated.findElement(By.id("searchInput"));         // dispara beforeFindElement/afterFindElement
```

A partir de ahí hay que usar `decorated`, no el `driver` original, para que los eventos se disparen. `onError` es especialmente útil: captura una pantalla automáticamente en cuanto **cualquier** llamada al driver lanza una excepción, sin tener que envolver cada test en un `try/catch` para conseguir lo mismo — muy valioso para diagnosticar por qué falló un test que se ejecutó en un entorno de CI al que no se tiene acceso directo.

9. Selenium Grid y ejecución remota

Hasta ahora, `FirefoxDriver` / `ChromeDriver` arrancan y controlan un navegador **en la misma máquina** donde corre el test. `RemoteWebDriver` habla en cambio con un **Selenium Grid**: un servidor (`hub`) que reparte la ejecución entre uno o varios `node` (que pueden estar en otras máquinas, con otros sistemas operativos o navegadores instalados).

BASH

```
# Arrancar el hub
java -jar selenium-server-<version>.jar hub

# Dar de alta un nodo (autodetecta los navegadores instalados)
java -jar selenium-server-<version>.jar node --detect-drivers true

# Panel de control
http://localhost:4444/ui/
```

JAVA

```
FirefoxOptions options = new FirefoxOptions();
options.setCapability(CapabilityType.PLATFORM_NAME, Platform.WINDOWS);

WebDriver driver = new RemoteWebDriver(new URL("http://localhost:4444"), options);
```

`*Options` (`FirefoxOptions`, `ChromeOptions`, `EdgeOptions` ...) describe qué navegador y en qué condiciones se pide al Grid — el propio `RemoteWebDriver` es idéntico independientemente del navegador real que acabe usándose; el resto del test (localizadores, esperas, aserciones) no cambia nada respecto a un driver local.

9.1. Combinándolo con tests en paralelo

Con `@Execution(ExecutionMode.CONCURRENT)` (ver el manual de JUnit) y un `@ParameterizedTest` que recorra una lista de navegadores, un mismo escenario se puede lanzar a la vez contra varios navegadores distintos del Grid:

JAVA

```
static Stream<Arguments> navegadores() {
    return Stream.of(Arguments.arguments("Firefox"), Arguments.arguments("Chrome"));
}

@ParameterizedTest
@MethodSource("navegadores")
void googleCargaCorrectamente(String navegador) {
    WebDriver driver = crearDriverPara(navegador); // RemoteWebDriver con las Options que toquen
    driver.get("http://www.google.es");
    assertEquals("Google", driver.getTitle());
    driver.quit();
}
```



Cada ejecución concurrente necesita su **propia** instancia de `WebDriver` (no se puede compartir un mismo `driver` entre hilos). Por eso el `driver` en este patrón se crea dentro del propio método de test, no en un `@BeforeAll` compartido — lo contrario acopla los tests entre sí exactamente igual que compartir estado mutable en JUnit.

10. Buenas prácticas (y errores comunes)

- **Preferir las esperas de Selenium a `Thread.sleep`**. Un `Thread.sleep` fijo es, a la vez, demasiado corto algunas veces (el test falla igualmente) y demasiado largo otras (el test tarda más de lo necesario). Ver Sincronización (esperas).
- **No depender de sitios externos reales** (`google.com`, `wikipedia.org` ...) para lo que se pueda probar contra una página propia y estable. El DOM, los textos legales de cookies, los formularios de un sitio de terceros cambian sin aviso y rompen el test sin que el código de la aplicación tenga ningún problema.
- **`click()` en checkboxes/radios alterna el estado**: comprobar siempre `isSelected()` antes de decidir si hace falta pulsar.
- **`getAttribute` vs `getProperty`**: el primero es el HTML fuente, el segundo el estado actual del DOM. Elegir el que corresponda según lo que realmente se quiera comprobar.
- **Encapsular cada pantalla en un Page Object** en cuanto varios tests tocan los mismos elementos — evita duplicar localizadores y concentra en un único sitio el impacto de un cambio en el HTML.
- **`driver.quit()` siempre en un `@AfterAll` / `@AfterEach`** salvo que se esté depurando el test a propósito y se quiera dejar el navegador abierto para inspeccionarlo.
- **Cada ejecución paralela necesita su propio `WebDriver`**. No compartir una instancia de `driver` entre hilos.
- **XPath para casos que CSS no puede expresar** (por ejemplo, seleccionar por el texto del elemento, o subir a un ancestro), CSS para todo lo demás — suele ser más corto y más rápido de evaluar.

11. Referencia rápida

Clase / método	Para qué
<code>By.id</code> / <code>By.name</code> / <code>By.className</code> / <code>By.tagName</code>	Localizadores directos por atributo.
<code>By.linkText</code> / <code>By.partialLinkText</code>	Localizar un <code><a></code> por su texto.
<code>By.xpath("...")</code> / <code>By.cssSelector("...")</code>	Localizadores expresivos para jerarquías/condiciones complejas.
<code>findElement</code> / <code>findElements</code>	Uno / todos los elementos que coinciden.
<code>click()</code> / <code>sendKeys()</code> / <code>clear()</code> / <code>submit()</code>	Interacción básica con un elemento.
<code>getText()</code> / <code>getAttribute()</code> / <code>getCssValue()</code>	Lectura de un elemento.
<code>Select</code>	Envoltorio para trabajar con <code><select></code> (simple o múltiple).
<code>Actions</code>	Interacciones compuestas: doble clic, arrastrar y soltar, teclas...

Clase / método	Para qué
JavaScriptExecutor	Ejecutar JavaScript en la página cargada.
driver.switchTo().alert()	Diálogos nativos del navegador.
driver.manage().timeouts().implicitlyWait(...)	Espera implícita, sobre toda la sesión.
WebDriverWait + ExpectedConditions	Espera explícita sobre una condición predefinida.
FluentWait	Espera explícita configurable (polling, excepciones ignoradas).
@FindBy + PageFactory.initElements	Page Object Model.
TakesScreenshot	Capturas de pantalla.
WebDriverListener + EventFiringDecorator	Reaccionar a eventos del driver (por ejemplo, capturar pantalla en cualquier error).
RemoteWebDriver + *Options	Ejecución contra un Selenium Grid.



Todos los conceptos de este manual se pueden practicar de forma guiada en la práctica Ej06_Selenium_4 del curso.